

Book 10: Final 24-Hour High-Yield Revision Pack

Last-Minute Cheatsheets & Active Recall Flashcards

IDFC FIRST Bank | Strategic Projects Developer (3+ Years Experience)

Personalized Hinglish Interview Preparation System • Production Ready

DSA Last-Minute Algorithmic Patterns Cheatsheet



The Top 14 Algorithmic Patterns Quick Reference

Pattern	Trigger in Problem Statement	Core Data Structure	Typical Complexity
1. Frequency Hash Map	"Find duplicates", "Frequency of elements", "Anagrams"	<code>collections.Counter</code> / <code>dict</code>	$O(N)$ Time, $O(N)$ Space
2. Two Pointers	"Sorted array", "Find pair summing to target", "Palindromes"	Two integer indices (<code>L</code> , <code>R</code>)	$O(N)$ Time, $O(1)$ Space
3. Sliding Window	"Contiguous subarray/substring", "Longest/Shortest with condition"	Two indices + Hash Map	$O(N)$ Time, $O(K)$ Space
4. Fast & Slow Pointers	"Cycle detection in linked list", "Find middle of linked list"	<code>slow</code> , <code>fast</code> pointers	$O(N)$ Time, $O(1)$ Space
5. Prefix Sum	"Subarray sum equals K", "Range sum queries"	Array / Running sum	$O(N)$ Time, $O(1)$ query
6. Monotonic Stack	"Next greater element", "Daily temperatures", "Stock span"	<code>list</code> as stack	$O(N)$ Time, $O(N)$ Space
7. Top K Elements	"Find K largest / K most frequent", "Running median"	<code>heapq</code> (Min-Heap / Max-Heap)	$O(N \log K)$ Time
8. Binary Search	"Sorted input", "Find peak", "Search rotated sorted array", "Min of Max"	<code>left</code> , <code>right</code> , <code>mid</code>	$O(\log N)$ Time, $O(1)$ Space
9. BFS (Level Order)	"Shortest path in unweighted graph/grid", "Level-by-level tree"	<code>collections.deque</code>	$O(V + E)$ Time, $O(V)$ Space
10. DFS (Backtracking)	"Generate all combinations/permutations/subsets", "Islands in grid"	Recursion / Call Stack	$O(2^N)$ or $O(N!)$ Time
11. Merge Intervals	"Overlapping intervals", "Meeting rooms schedule"	Sort by <code>start_time</code>	$O(N \log N)$ Time
12. Dynamic Programming	"Overlapping subproblems", "Max/Min path", "Number of ways"	1D / 2D table or memo dict	$O(N)$ or $O(N \times M)$
13. Greedy	"Interval scheduling", "Jump game", "Local optimum yields global"	Priority Queue or Sort	$O(N \log N)$ Time
14. Union-Find (DSU)	"Connected components", "Detect cycle in undirected graph"	<code>parent</code> and <code>rank</code> arrays	$O(\alpha(N))$ approx $O(1)$

Python High-Yield Syntax Snippets

```
# 1. Monotonic Deque for O(1) pops and appends:
from collections import deque
q = deque()
q.append(1)
q.popleft() # O(1) vs list.pop(0) which is O(N)!

# 2. Min-Heap & Max-Heap:
import heapq
min_h = []
heapq.heappush(min_h, 10)
val = heapq.heappop(min_h)
# Max-heap: push negative values
heapq.heappush(min_h, -10)
val = -heapq.heappop(min_h)

# 3. Safe Binary Search Mid Calculation:
mid = left + (right - left) // 2

# 4. In-Place Reversal:
def reverse_range(arr, l, r):
    while l < r:
        arr[l], arr[r] = arr[r], arr[l]
        l += 1
        r -= 1

# 5. Defaultdict without KeyError:
from collections import defaultdict
graph = defaultdict(list)
counts = defaultdict(int)
```

SQL & DBMS Last-Minute Interview Cheatsheet

SQL Logical Execution Order

1. FROM & JOIN (Combines tables)
 2. ON (Filters join rows)
 3. WHERE (Filters base rows before grouping - NO AGGREGATES!)
 4. GROUP BY (Aggregates rows into buckets)
 5. HAVING (Filters aggregated groups)
 6. SELECT (Calculates expressions & column aliases)
 7. DISTINCT (Deduplicates output)
 8. ORDER BY (Sorts results)
 9. LIMIT / OFFSET (Paginates results)
-

Window Functions Cheat Sheet

```
-- General Syntax:
FUNCTION() OVER (
    PARTITION BY category_col
    ORDER BY sort_col
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)

-- Key Ranking Differences:
ROW_NUMBER() -- 1, 2, 3, 4 (Always unique sequential integers)
RANK()        -- 1, 2, 2, 4 (Ties get same rank, subsequent rank skipped)
DENSE_RANK()  -- 1, 2, 2, 3 (Ties get same rank, NO rank skipped - Top N salaries!)

-- Navigation:
LAG(col, 1)   -- Reads value from previous row (Gap / time difference)
LEAD(col, 1)  -- Reads value from next row

-- Running Total (Explicit Frame is faster than default RANGE!):
SUM(amount) OVER (
    PARTITION BY account_id
    ORDER BY created_at
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)
```

ACID & Database Concurrency Cheat Sheet

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Default In
Read Uncommitted	✗ Allowed	✗ Allowed	✗ Allowed	—
Read Committed	✓ Prevented	✗ Allowed	✗ Allowed	Postgres / Oracle
Repeatable Read	✓ Prevented	✓ Prevented	✓ Prevented in Postgres	MySQL (InnoDB)
Serializable	✓ Prevented	✓ Prevented	✓ Prevented	SQLite / High Security

Locking Commands

```
-- Pessimistic Lock (Blocks other readers and writers):
SELECT * FROM accounts WHERE account_id = 101 FOR UPDATE;

-- Pessimistic Lock (Fail immediately if locked):
SELECT * FROM accounts WHERE account_id = 101 FOR UPDATE NOWAIT;

-- Task Queue Processing (Skip locked rows to prevent worker pileup):
SELECT * FROM job_queue WHERE status = 'PENDING' LIMIT 1 FOR UPDATE SKIP LOCKED;

-- Optimistic Lock Check:
UPDATE accounts
SET balance = balance - 100, version = version + 1
WHERE account_id = 101 AND version = :read_version;
```

B+ Tree Indexing Rules

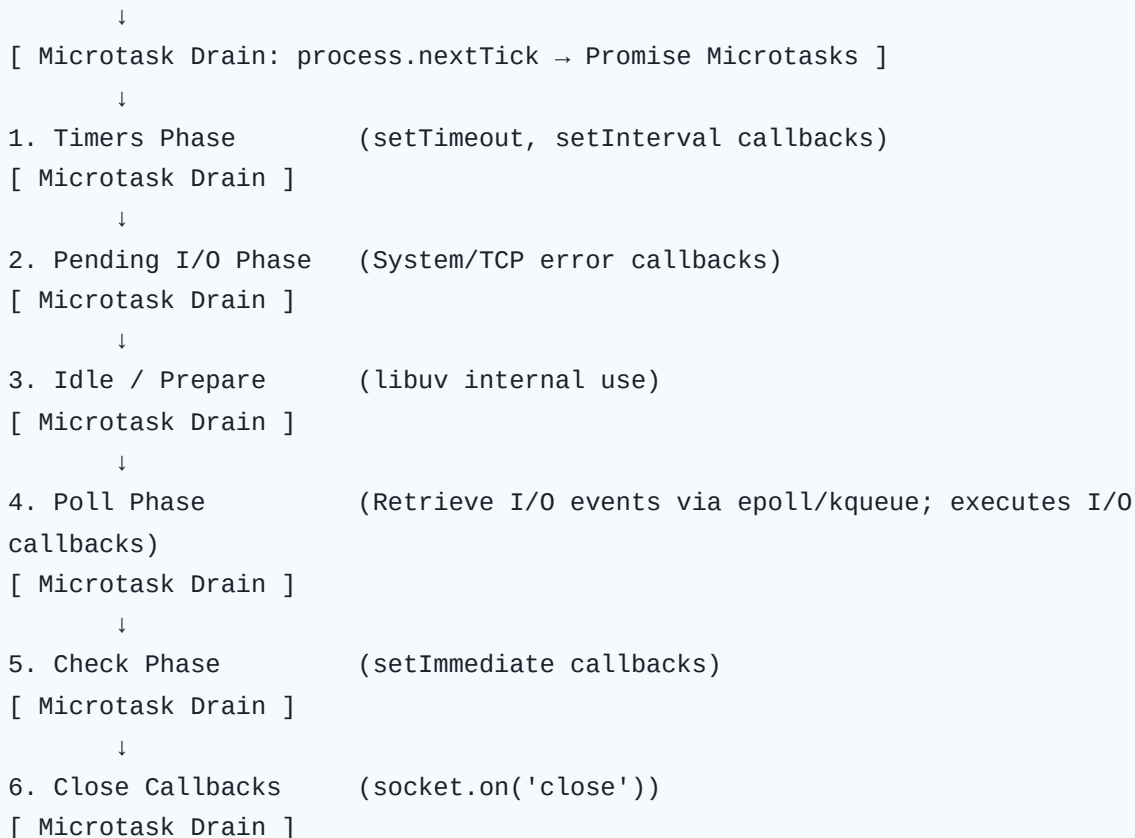
- Leftmost Prefix Rule:** If composite index is on (A, B, C) :
- WHERE A = 1 \$to\$ **Index Scan (Yes)**
- WHERE A = 1 AND B = 2 \$to\$ **Index Scan (Yes)**
- WHERE B = 2 AND C = 3 \$to\$ **Sequential Scan (No index used!)**
- Range Breaks Indexing:** If query has WHERE A = 1 AND B > 5 AND C = 10 :
- Column A uses index seek.
- Column B uses index range scan.

8. Column **c** **cannot** use the index; must be filtered after scan!
9. **Clustered Index:** Leaf nodes contain full data rows (1 per table).
10. **Secondary Index:** Leaf nodes contain indexed column + Primary Key pointer.
11. **Covering Index:** Index contains *all* projected columns, bypassing table data pages entirely.

Node.js & Backend Architecture Last-Minute Cheatsheet

libuv Event Loop Phase Order

Synchronous Call Stack (V8)



Key Differences & Interview Gotchas

1. `process.nextTick()` vs `setImmediate()` :
2. `process.nextTick()` is **not** part of libuv; executes immediately after current synchronous code. Infinite recursion freezes the server!
3. `setImmediate()` runs during the libuv **Check phase**.
4. Inside an I/O callback, `setImmediate` **always runs before** `setTimeout(0)` .
5. **What uses the libuv Worker Thread Pool (`UV_THREADPOOL_SIZE = 4`)?**
6. **Uses Thread Pool:** `fs` (file system), `crypto` (`pbkdf2` , `scrypt`), `zlib` (compression), `dns.lookup` .

7. **Does NOT use Thread Pool:** Network sockets (`http` , `https` , `tls` , `net`), `dns.resolve` (handled non-blocking via OS `epoll` / `kqueue`).

8. **Streams & Backpressure:** ``javascript // Anti-Pattern (Leaks RAM when client network is slow):
`fs.readFile('huge.csv', (err, data) => res.send(data));`

// Production Pattern with Backpressure (Stream pipeline): `const { pipeline } = require('stream/promises'); await pipeline(fs.createReadStream('huge.csv'), res); ```

Memory Leak Detection in Node.js

- **Tool:** Chrome DevTools (`node --inspect`) or `v8.writeHeapSnapshot()` .
- **Top 3 Culprits:**
 - Unbounded global caching objects without TTL/eviction.
 - Unremoved event listeners (`emitter.on` inside request handlers).
 - Stale closures retaining large outer function scope objects.
- **Metric to watch:** `process.memoryUsage().heapUsed` vs `heapTotal` .

Clustering vs Worker Threads

Dimension	Cluster Module (<code>cluster</code>)	Worker Threads (<code>worker_threads</code>)
Model	Multi-Process (Forking)	Multi-Thread within single process
Memory	Isolated memory space per core	Shared memory via <code>SharedArrayBuffer</code>
Port Sharing	Shares same server port via master proxy	Run within existing master process
Best Used For	Scaling web servers across CPU cores	Heavy CPU tasks (image resize, crypto)

System Design Last-Minute Interview Cheatsheet

Numbers Every Engineer Must Know (Latencies)

L1 Cache Reference:	~0.5 ns
Branch Mispredict:	~5 ns
L2 Cache Reference:	~7 ns
Mutex Lock/Unlock:	~25 ns
Main Memory (RAM) Reference:	~100 ns
Compress 1KB with Snappy:	~3 µs
Send 1KB over 1Gbps Network:	~10 µs
Read 1MB Sequentially from SSD:	~1 ms
Disk Seek (HDD):	~10 ms
Send Packet Across Datacenter:	~0.5 ms
Round Trip (Mumbai to US-East):	~150 ms

High Availability & Uptime SLAs

Availability Target	Annual Allowed Downtime	Daily Allowed Downtime	Typical Tier
99% ("Two Nines")	3.65 days	14.4 minutes	Internal testing tools
99.9% ("Three Nines")	8.76 hours	43.2 seconds	Standard SaaS apps
99.99% ("Four Nines")	52.6 minutes	4.32 seconds	High-priority API gateways
99.999% ("Five Nines")	5.26 minutes	0.43 seconds	Core Banking Solution & Payment Rails

Redis Caching Strategies & Defenses

- 1. Cache-Aside (Lazy Loading):** App reads cache. On miss, reads DB, writes to cache, returns. (Most common).
- 2. Write-Through:** App writes to cache; cache synchronously writes to DB before acknowledging.

3. **Write-Behind (Write-Back):** App writes to cache; cache asynchronously batches writes to DB. (High write throughput, risk of data loss on crash).
 4. **Cache Penetration Defense:** Querying keys that don't exist in DB.
 5. *Fix:* Cache `null` with short TTL, or use a **Bloom Filter** at the gateway.
 6. **Cache Stampede / Breakdown:** Hot key with high QPS expires; thousands of requests hit DB simultaneously.
 7. *Fix:* Mutual exclusion lock (Mutex in Redis) or probabilistic early expiration (XFetch algorithm).
-

Apache Kafka Quick Architecture

- **Topic:** Logical stream of messages.
- **Partition:** Ordered, append-only log of immutable messages. The unit of parallelism.
- **Partition Key:** Messages with the same key (e.g., `account_id`) **always go to the same partition**, guaranteeing chronological ordering per account.
- **Consumer Group:** Multiple workers reading a topic. Each partition is consumed by strictly **one** consumer within a group.
- **Replication Factor:** `N` copies across brokers (typical in banking: 3 replicas with `min.insync.replicas = 2` and `acks = all` for zero message loss).

Active Recall Flashcards for IDFC FIRST Bank Interview

Banking & FinTech Flashcards

Flashcard B1

- **Q:** What is the core rule of double-entry bookkeeping?
 - **A:** Every transaction must record at least one Debit and one Credit such that: $\sum \text{Debits} = \sum \text{Credits}$ The net financial sum of any balanced transaction entry is strictly zero.
-

Flashcard B2

- **Q:** What is an Idempotency Key in a payment API?
 - **A:** A unique client-generated token (typically UUIDv4) transmitted in headers (`X-Idempotency-Key`). It guarantees that duplicate retries of an identical request will execute the underlying business logic and financial debit exactly once, returning the cached outcome on subsequent calls.
-

Flashcard B3

- **Q:** What are the 4 main payment rails in India?
 - **A:**
 - **UPI:** Instant real-time gross settlement 24/7/365, VPA-based, capped at ₹1-5 Lakh.
 - **IMPS:** Instant 24/7 account-based settlement, capped at ₹5 Lakh.
 - **NEFT:** Deferred Net Settlement in 30-minute batches 24/7, no upper limit.
 - **RTGS:** Real-Time Gross Settlement for high-value transactions (minimum ₹2 Lakh).
-

Flashcard B4

- **Q:** What does PCI-DSS strictly forbid regarding card security?
 - **A:** Never store Sensitive Authentication Data (SAD) after authorization—specifically the 3-digit **CVV/CVC** or raw card PIN, even if encrypted.
-

SQL & DBMS Flashcards

Flashcard S1

- **Q:** What does ACID stand for?
 - **A:**
 - **Atomicity:** All statements commit or all rollback (WAL undo logs).
 - **Consistency:** All integrity constraints and invariants remain valid.
 - **Isolation:** Concurrent transactions execute without mutual interference (2PL/MVCC).
 - **Durability:** Committed transactions survive system crashes (WAL `fsync`).
-

Flashcard S2

- **Q:** What is the difference between `RANK()` and `DENSE_RANK()` ?
 - **A:** In ties:
 - `RANK()` assigns identical ranks and skips subsequent values (\$1, 2, 2, 4\$).
 - `DENSE_RANK()` assigns identical ranks and does NOT skip subsequent values (\$1, 2, 2, 3\$).
-

Flashcard S3

- **Q:** What is a Write-Ahead Log (WAL)?
 - **A:** An append-only disk log where transaction changes must be recorded and flushed via sequential `fsync()` *before* any actual data pages are modified on disk storage.
-

Flashcard S4

- **Q:** Why is `NOT IN` dangerous when subquery results contain `NULL` ?
 - **A:** Due to SQL's three-valued logic, `x != NULL` evaluates to `UNKNOWN` . If any subquery row is `NULL` , the entire `NOT IN` conjunction evaluates to `UNKNOWN` , causing the query to return zero rows silently. Use `NOT EXISTS` instead.
-

Node.js & JavaScript Flashcards

Flashcard N1

- **Q:** What is the execution difference between `process.nextTick()` and `setImmediate()` ?
- **A:**
- `process.nextTick()` belongs to the process microtask queue; it runs immediately when the active call stack clears, before the event loop advances.
- `setImmediate()` belongs to the libuv **Check phase** (macrotask). Inside an I/O callback, `setImmediate` always runs before `setTimeout(0)` .

Flashcard N2

- **Q:** Which Node.js asynchronous operations use the 4-thread worker pool?
 - **A:** Only `fs`, `crypto`, `zlib`, and `dns.lookup`. Network I/O (`http`, `https`, `tls`, `net`) uses non-blocking OS kernel notifications (`epoll` / `kqueue`) and bypasses the thread pool entirely.
-

Flashcard N3

- **Q:** What causes a Stale Closure in React?
 - **A:** When a hook closure (like `useEffect` or `useCallback`) captures state or prop values from an earlier render because those values were omitted from the dependency array.
-



System Design Flashcards

Flashcard D1

- **Q:** What is the CAP theorem trade-off in banking?
 - **A:** Banking systems are **CP systems** (Consistency & Partition Tolerance). In a network partition, the system rejects writes or degrades availability to guarantee that financial balances are never duplicated or double-spent.
-

Flashcard D2

- **Q:** How do you guarantee message ordering in Apache Kafka?
- **A:** Kafka guarantees strict message ordering **only within a single partition**. By setting the message key to `account_id`, all events for that account are routed to the same partition and processed strictly in chronological order.